

Fahranweisung (Drive Instructions) — Vollständige Dokumentation

Dieses Dokument fasst die Backend- und Frontend-Dokumentation zur Fahranweisung in der New Dispo App zusammen. Es enthält Informationen zu Endpoints, Datenverträgen, Authentifizierung, Frontend-Darstellung sowie die Zuordnung der TMS-Bridge-Operationen zu den zugrunde liegenden Datenbank-Views, -Funktionen und -Prozeduren.

Swagger / OpenAPI: Die vollständige Backend API-Spezifikation liegt bei. Swagger UI ist in den Umgebungen `Local` und `Development` unter `/swagger` erreichbar und bietet Bearer-Token-Authentifizierung zum Testen.

Inhaltsverzeichnis

- 1. [Authentifizierung](#)
- 2. [Backend API Endpoints](#)
- 3. [Datenverträge \(Data Contracts\)](#)
- 4. [Frontend-Darstellung](#)
- 5. [TypeScript-Interfaces](#)
- 6. [TMS Bridge — GraphQL Mutations & Queries](#)
- 7. [TMS Bridge — Datenbank-Zuordnung](#)
- 8. [Quellcode-Referenzen](#)

1. Authentifizierung

Mechanismus: Keycloak OpenID Connect (OAuth 2.0 / Bearer Token)

Required Headers on Every Request

Header	Beschreibung
Authorization	Bearer {JWT token} aus Keycloak
Content-Type	application/json
Database-Identifizier	Identifiziert die Ziel-Datenbank (z.B. <code>main</code> , aus <code>localStorage</code>)

- Die Session wird aktiv überwacht. Token-Erneuerung erfolgt 50 Sekunden vor Ablauf. Bei fehlgeschlagener Erneuerung wird der Nutzer ausgeloggt.

Relevante Frontend-Dateien

Datei	Zweck
<code>libs/nagel-services/src/lib/keycloakService/keycloak.service.ts</code>	Token-Verwaltung, Init, Renewal, Logout
<code>libs/nagel-services/src/lib/requestService/request.service.ts</code>	Fügt Bearer Token + Database-Identifizier zu allen Requests hinzu
<code>apps/nagel-cal-disposition/src/app/app.config.ts</code>	APP_INITIALIZER mit Keycloak, HTTP-Interceptor-Setup

Datei	Zweck
apps/nagel-cal-disposition/src/app/app.component.ts (Zeilen 106-145)	Session-Activity-Monitoring & Token-Refresh

2. Backend API Endpoints

Alle Endpoint-Konstanten sind definiert in:
apps/nagel-cal-disposition/src/app/configuration/consts/endpoints.ts

Die Base-URL kommt aus der Environment-Config (z.B. http://localhost:5101 für Dev).

2.1 Fahrenweisung / Tourpunkte

Methode	Endpoint	Beschreibung
GET	/api/pickup-planning/transportorders/{id}/drive-instructions	Alle Tourpunkte (Fahrenweisung) eines Transportauftrags abrufen
POST	/api/pickup-planning/transportorders/tourpoint	Neuen Tourpunkt anlegen
PUT	/api/transportorders/tourpoints/{tourPointId}	Bestehenden Tourpunkt bearbeiten
DELETE	/api/pickup-planning/transportorders/tourpoint	Tourpunkt löschen
PUT	/api/pickup-planning/tourpoints/reorder	Tourpunkte umsortieren (Drag & Drop)
PATCH	/api/transportorders/tourpoint/{tourPointId}/tournumber	Kunden-Tournummer setzen
POST	/api/pickup-planning/transportorders/graph-tour-points	Graph-Tourpunkte für Visualisierung abrufen

2.2 Ladeintervalle (Loading Intervals)

Methode	Endpoint	Beschreibung
PUT	/api/transportorders/tourpoints/{tourPointId}/loading-interval	Ladeintervall (Zeitfenster) setzen
DELETE	/api/transportorders/tourpoints/{tourPointId}/loading-interval	Ladeintervall entfernen

2.3 Ladereferenz (Loading Reference)

Methode	Endpoint	Beschreibung
PUT	/api/transportorders/tourpoints/{tourPointId}/loading-reference	Ladereferenz setzen

2.4 Zuweisungen & Ladereihenfolge (Assignments / Loading Sequence)

Methode	Endpoint	Beschreibung
POST	/api/pickup-planning/lotassignments/from-leg	Sendung (Leg) einem Transportauftrag zuweisen
POST	/api/pickup-planning/lotassignments/from-lot	Partie (Lot) einem Transportauftrag zuweisen
PUT	/api/pickup-planning/lotassignments/reorder	Partien umsortieren (Ladereihenfolge der Lots)
PUT	/api/pickup-planning/legs/reorder	Sendungen innerhalb einer Partie umsortieren
PUT	/api/pickup-planning/unassign	Legs/Lots von einem Transportauftrag lösen

Methode	Endpoint	Beschreibung
PATCH	/api/pickup-planning/legs/{legId}/stays-loaded	Sendung als "bleibt geladen" markieren

2.5 Transportauftrags-Details

Methode	Endpoint	Beschreibung
GET	/api/transportorders/{orderNumber}	Vollständige Transportauftragsdetails
POST	/api/transportorders/paged	Paginierte Transportauftragsliste

3. Datenverträge (Data Contracts)

3.1 Response-Struktur: Drive Instructions

```
DriveInstructionsTourPointCardDto
├─ tourpointId, type (1=pickup, 3=delivery), sequenceNumber
├─ name, street, country, postalCode, city
├─ plannedArrivalTime, plannedDepartureTime
├─ weight, floorPalletSpaces, volumePalletSpaces
├─ uniqueClientsCount, uniqueTrafficFlowsCount
├─ productGroups: List<string>
├─ lotAssignmentCards[]
│   └─ lotAssignmentId, number, legsCount, pickupTourPointOrder
│       └─ legCards[]
│           └─ legId, shipmentNumber, order
│               └─ name, street, country, city, zipCode
│                   └─ weight, volumePalletSpaces, floorPalletSpaces
│                       └─ deliveryDateFrom/To, pickupDateFrom/To, fixedDeliveryDate
│                           └─ staysLoaded: bool
│                               └─ trafficIcon (ArrowDown=loading, ArrowUp=unloading)
```

3.2 Tourpunkt-Typen

Definiert in apps/nagel-cal-disposition/src/app/utils/tourPointsUtils.ts :

Wert	Typ
1	Pickup (Ladestelle)
3	Delivery (Entladestelle)
5	Exchange of Carrier (Unternehmerwechsel)
8	Border Crossing (Grenzübergang)
9	Customs (Zoll)
11	Start
12	Finish

Relation-Typen für Positionierung: 0 =first, 1 =last, 2 =after, 3 =before

3.3 Request/Response Bodies

Tourpunkt anlegen (POST /api/pickup-planning/transportorders/tourpoint)

Request:

```
{
  "transportOrderId": 123,
  "tourPointType": 11,
  "name1": "string",
  "productType": null,
  "tourNumber": "string | null",
  "country": "string | null",
  "postalCode": "string | null",
  "city": "string | null",
  "streetAndHouseNumber": "string | null",
  "district": "string",
  "deliveryDateFrom": "ISO date | null",
  "deliveryDateTo": "ISO date | null",
  "deliveryTimeFrom": "ISO date | null",
  "deliveryTimeTo": "ISO date | null"
}
```

Response:

```
{
  "IsTourpointAdded": true,
  "TourpointId": 456
}
```

Tourpunkt bearbeiten (PUT /api/transportorders/tourpoints/{tourPointId})

Request:

```
{
  "name1": "string",
  "tourNumber": "string",
  "country": "string",
  "postalCode": "string",
  "city": "string",
  "streetAndHouseNumber": "string",
  "district": "string"
}
```

Response:

```
{
  "IsTourpointEdited": true
}
```

Tourpunkt löschen (DELETE /api/pickup-planning/transportorders/tourpoint)

Request:

```
{
  "TourPointId": 456,
  "TransportOrderId": 123,
  "Mode": 11
}
```

Response:

```
{
  "IsTourpointDeleted": true
}
```

Tourpunkte umsortieren (PUT /api/pickup-planning/tourpoints/reorder)

Request:

```
{
  "SourceTransportOrderTix": 123,
  "SourceTourpointId": 456,
  "DestinationTourpointId": 789,
  "RelationType": 2,
  "Mode": null
}
```

Kunden-Tournummer setzen (PATCH /api/transportorders/tourpoint/{tourPointId}/tournumber)

Request:

```
{
  "tourPointId": 456,
  "tourNumber": "string | null",
  "personNumber": 0
}
```

Ladeintervall setzen (PUT /api/transportorders/tourpoints/{tourPointId}/loading-interval)

Request:

```
{
  "StartTime": "ISO datetime | null",
  "EndTime": "ISO datetime | null"
}
```

Response:

```
{
  "isStartTimeSet": true,
  "isEndTimeSet": true
}
```

Ladeintervall entfernen (DELETE /api/transportorders/tourpoints/{tourPointId}/loading-interval)

Response:

```
{
  "isDeleted": true
}
```

Ladereferenz setzen (PUT /api/transportorders/tourpoints/{tourPointId}/loading-reference)

Request:

```
{
  "LoadingReference": "Gate 5"
}
```

Response:

```
{
  "isLoadingReferenceSet": true
}
```

Partien umsortieren (PUT /api/pickup-planning/lotassignments/reorder)

Request:

```
{
  "LotAssignmentId": "guid",
  "NewPickupTourPointOrder": 1,
  "TransportOrderId": 123
}
```

Sendungen innerhalb Partie umsortieren (PUT /api/pickup-planning/legs/reorder)

Request:

```
{
  "LotAssignmentId": "guid",
  "LegId": "guid",
  "NewOrder": 1
}
```

"Bleibt geladen" markieren (PATCH /api/pickup-planning/legs/{legId}/stays-loaded)

Query Parameter: staysLoadedValue: bool

Sendung zuweisen (POST /api/pickup-planning/lotassignments/from-leg)

Request:

```
{
  "transportOrderId": 123,
  "legId": "guid",
  "lotAssignmentId": "guid",
  "destinationTourPointId": 456,
  "relationType": 2
}
```

Partie zuweisen (POST /api/pickup-planning/lotassignments/from-lot)

Request:

```
{
  "transportOrderId": 123,
  "lotId": "guid",
  "destinationTourPointId": 456,
  "relationType": 2
}
```

Legs/Lots lösen (PUT /api/pickup-planning/unassign)

Request:

```
{
  "LotAssignmentIds": ["guid1", "guid2"],
  "LegIds": ["guid3"],
  "TransportOrderId": 123
}
```

Response:

```
{
  "Result": true,
  "Value": {
    "LotsResponse": [{ "Success": true, "Id": "guid1", "Error": null }],
    "LegsResponse": [{ "Success": true, "Id": "guid3", "Error": null }]
  }
}
```

Graph-Tourpunkte abrufen (POST /api/pickup-planning/transportorders/graph-tour-points)

Request:

```
{
  "TransportOrderIds": [123, 456]
}
```

4. Frontend-Darstellung

4.1 Architektur-Überblick

Das Frontend ist eine **Angular**-Anwendung (Nx Monorepo) unter apps/nagel-cal-disposition/ . Die Fahrplanweisung wird in zwei Kontexten dargestellt:

- 1. **Transport Order Slider** (Planning-View) — Tourpunkte mit Partien und Sendungen als verschachtelte Karten
- 2. **Transport Order Details** — Detailansicht eines Transportauftrags mit editierbaren Tourpunkt-Daten

4.2 Zentrale Services

Service	Datei	Verantwortung
Manage Tour Points	apps/.../services/manage-tour-points.service.ts	CRUD-Operationen auf Tourpunkten
Drive Instructions Drawer	apps/.../services/drive-instructions-drawer.service.ts	Drawer-Steuerung & Daten laden
Reorder Drag & Drop	apps/.../services/reorder-drag-and-drop.service.ts	Tourpunkt-Umsortierung per D&D

Service	Datei	Verantwortung
Assignment Requests	apps/.../services/assignment-requests.service.ts	Leg/Lot-Zuweisungen
Graph Tour Points	apps/.../services/graph-tour-points.service.ts	Graph-Visualisierung
Planning Drag & Drop	apps/.../services/planning-drag-and-drop.service.ts	D&D in der Planungsansicht

4.3 Datenmodell-Dateien

Modell	Datei
TourPointDetails, Loading Interval DTOs	apps/.../models/orderDetails.ts
Create/Edit/Delete Tour Point Types	apps/.../models/tourPointTypes.ts
TourPointConfig, LotTourPointConfig, LegTourPointConfig	apps/.../models/planningPageTypes.ts
Assignment Request/Response Types	apps/.../models/assignmentsTypes.ts
Tour Point Type Constants & Utilities	apps/.../utils/tourPointsUtils.ts
Endpoint-Konstanten	apps/.../configuration/consts/endpoints.ts

5. TypeScript-Interfaces

5.1 TourPointDetails

Definiert in `apps/nagel-cal-disposition/src/models/orderDetails.ts` (Zeilen 197-243):


```

export interface TourPointDetails {
  addressId: number;
  addressType: string;
  adviceAmount: number;
  branch: number;
  city: string;
  company: number;
  country: string;
  district: string;
  driveInstruction: string;
  floorPalletSpaces: number;
  frozenAmount: number;
  kind: number;
  latitude: number;
  longitude: number;
  match: string;
  name1: string | null;
  name2: string | null;
  name3: string | null;
  packageAmount: number;
  personId: string;
  personIndex: number;
  personNumber: number | null;
  personText: string | null;
  personType: string;
  plannedArrivalTime: string | null;
  plannedDepartureTime: string | null;
  targetEndTime: string | null;
  targetStartTime: string | null;
  postalCode: string;
  sequenceNumber: number;
  serviceArea: string;
  shipmentAmount: number;
  street: string;
  tourpointId: number;
  transportOrderId: number;
  type: number;
  volumePalletSpaces: number;
  weight: number;
  lotAssignmentIds: string[];
  tourNumber: string | null;
  formKey: number;
  isNew?: boolean;
  isEditableTourPoint?: boolean;
  distance: number | null;
  totalDuration: number | null;
}

```

5.2 TourPointConfig (Planning-View)

Definiert in `apps/nagel-cal-disposition/src/models/planningPageTypes.ts` (Zeilen 396-420):

```
export interface TourPointConfig {
  tourpointId: number;
  type: number;
  sequenceNumber: number;
  trafficIcon: TrafficIconType;
  commaSeparatedLotAssignmentNumbers: string;
  name: string;
  street: string;
  country: string;
  postalCode: string;
  city: string;
  locationIdentifier: string;
  plannedArrivalTime: null;
  plannedDepartureTime: string | null;
  weight: number | null;
  floorPalletSpaces: number;
  floorPalletSpacesIdentifier: string;
  volumePalletSpaces: number;
  volumePalletSpacesIdentifier: string;
  uniqueClientsCount: number;
  uniqueTrafficFlowsCount: string;
  productGroups: object[];
  lotAssignmentCards: LotTourPointConfig[];
  infoChips?: InfoLotChip[];
}
```

5.3 LotTourPointConfig (Partien am Tourpunkt)

```
export interface LotTourPointConfig {
  lotAssignmentId: string;
  number: number;
  legsCount: number;
  trafficIcon: string;
  pickupTourPointOrder: number;
  legCards: LegTourPointConfig[];
  checked?: boolean;
}
```

5.4 LegTourPointConfig (Sendungen innerhalb einer Partie)

```
export interface LegTourPointConfig {
  legId: string;
  shipmentNumber: number;
  order: number;
  name: string | null;
  country: string | null;
  city: string | null;
  street: string | null;
  zipCode: string | null;
  locationIdentifier: string | null;
  weight: number | null;
  volumePalletSpaces: number | null;
  volumePalletSpacesIdentifier: string | null;
  floorPalletSpaces: number | null;
  floorPalletSpacesIdentifier: string | null;
  destinationCountry: string | null;
  consigneeServiceArea: string | null;
  serviceAreaIdentifier: string | null;
  fixedDeliveryDate: string | null;
  deliveryDateFrom: string | null;
  deliveryDateTo: string | null;
  pickupDateFrom: string | null;
  pickupDateTo: string | null;
  trafficIcon: string;
  infoChips: InfoLotChip[];
  checked?: boolean;
  staysLoaded?: boolean;
}
```

6. TMS Bridge — GraphQL Mutations & Queries

Mehrere Backend-Endpoints delegieren Schreiboperationen an die TMS Bridge (Disposition-Abstraction-Layer) via GraphQL. Die Bridge-Base-URL ist pro Umgebung konfiguriert (z.B. `http://localhost:5158/bridge/`). Das Backend leitet den Bearer-Token des Aufrufers an die Bridge weiter.

6.1 Mutations-Übersicht (vom Backend aufgerufen)

GraphQL Mutation	Backend Endpoint	Response Field
callMoveTourpoint	PUT /api/pickup-planning/tourpoints/reorder	isTourpointMoved
callAddTourpoint	POST /api/pickup-planning/transportorders/tourpoint	isTourpointAdded , tourpointId
callDeleteTourpoint	DELETE /api/pickup-planning/transportorders/tourpoint	isTourpointDeleted
callEditTourpoint	PUT /api/transportorders/tourpoints/{id}	isTourpointEdited
callSetLoadingReference	PUT /api/transportorders/tourpoints/{id}/loading-reference	isLoadingReferenceSet
callSetTargetLoadingStartTime	PUT /api/transportorders/tourpoints/{id}/loading-interval	isStartTimeSet
callSetTargetLoadingEndTime	PUT /api/transportorders/tourpoints/{id}/loading-interval	isEndTimeSet
callRemoveLoadingIntervals	DELETE /api/transportorders/tourpoints/{id}/loading-interval	isDeleted

GraphQL Mutation	Backend Endpoint	Response Field
callStaysLoaded	PATCH /api/pickup-planning/legs/{id}/stays-loaded	isStaysLoadedSet , tmsLegId
callSetCustomerTourNumber	PATCH /api/transportorders/tourpoint/{id}/tournumber	isCustomerTourNumberSet

6.2 Weitere TMS Bridge Mutations (vollständig)

GraphQL Mutation	Zweck
callCreateTransportOrderFromLeg	TA aus einem Leg erstellen
callCreateTransportOrderFromLot	TA aus einer Sendung erstellen (<i>obsolete</i>)
callCreateAndAddLeg	Leg erstellen und direkt einem TA hinzufügen
callDeleteTransportOrder	Transportauftrag löschen
callRemoveLeg	Leg aus TA entfernen
callRemoveShipmentFromTransportOrder	Sendung aus TA entfernen
callAssignLotToTransportOrder	Lot einem TA zuweisen (<i>obsolete</i>)
callAssignVehicle	Fahrzeug einem TA zuweisen
callAddTrailer	Anhänger einem TA zuweisen
callSetVehicleAttributes	Fahrzeugattribute setzen
callSetPresetTemp	Vorgabetemperatur setzen
callSetParticipant	Beteiligten setzen
callSetXServerDto	XServer DTO setzen

6.3 TMS Bridge Queries

GraphQL Query	Zweck
getPagedTransportOrders	Paginierte TA-Liste
getTransportOrders	TA-Liste (unpaginiert)
getPagedFilteredTransportOrders	Gefilterte paginierte TA-Liste
getFilteredTransportOrders	Gefilterte TA-Liste
getPickupPlanningTransportOrders	TA für Pickup-Planung
getGroupedTransportOrdersAsync	Gruppierte TA-Übersicht
getTransportOrderDetails	TA-Detaildaten
getTourpoints	Tourpunkte eines TA
getLegs	Legs (Lade-/Entladepunkte)
getShipments / getEBVShipment	Sendungsdaten
getStaysLoaded	"Bleibt geladen"-Status eines Legs
getVehicles	Fahrzeugdaten
getPresetsTemp	Temperaturvorgaben

GraphQL Query	Zweck
getFreightExchangeTourpoints	Frachtbörsen-Tourpunkte
getBranchAddresses	Niederlassungsadressen
getPagedLocationsEntities	Standortdaten (paginiert)
getPersonEntities / getPersonPagedEntities	Personenstammdaten
getPagedParticipants	Beteiligte (paginiert)
getTourpointClientCommunication	Kundenkommunikation zu Tourpunkten
getTourpointTargetDatesFieldValues	Ziel-Ladedaten
getContactDetailsFieldValues	Kontaktdetails
getBorderoCartages / getRollkartCartages	Kartage-Daten
getXserverDto	XServer-Daten

7. TMS Bridge — Datenbank-Zuordnung

7.1 Queries → Datenbank-Views

GraphQL Query	Datenbank-View
getPagedTransportOrders / getTransportOrders	v_dis_transportorder
getPagedFilteredTransportOrders / getFilteredTransportOrders / getTransportOrdersFieldValues	v_dis_transportorder_filter
getPickupPlanningTransportOrders / getGroupedTransportOrdersAsync	v_dis_transportorder_pickupplanning
getTourpoints	v_dis_to_tourpoint
getLegs	v_dis_leg
getShipments	v_dis_shipment_all
getPresetsTemp	v_dis_transportorder_presettemp
getFreightExchangeTourpoints	v_dis_freight_exchange_tourpoints
getBranchAddresses	v_dis_branch_address
getVehicles	v_dis_vehicle
getPagedParticipants	v_dis_participant
getContactDetailsFieldValues	v_dis_contactdetails
getTourpointTargetDatesFieldValues	v_dis_tourpoint_targetdates
getTourpointClientCommunication	v_dis_tourpoint_clientcommunication

7.2 Queries → Datenbank-Funktionen

GraphQL Query	Datenbank-Funktion
getTransportOrderDetails	pdis_transportorderdto.get()

GraphQL Query	Datenbank-Funktion
getStaysLoaded	pdis_leg.getstaysloadedstatus(p_legid)
getXserverDto	pdis_transportorder.getxserverdto(sid)

7.3 Queries → Datenbank-Tabellen (LINQ Joins)

GraphQL Query	Tabellen
getBorderoCartages	bordero + sendung + person (Multi-Table JOIN)
getRollkartCartages	rollkart + sendung + person (Multi-Table JOIN)
getPagedLocationsEntities	ort
getPersonEntities / getPersonPagedEntities	person
getSendungEntities	sendung
getSenZuordEntities	sen_zuord
getSenLsRef	sen_ls_ref
getSenRef	sen_ref

7.4 Mutations → Datenbank-Prozeduren (Fahranweisungs-relevant)

GraphQL Mutation	Datenbank-Prozedur	Parameter
callMoveTourpoint	pdis_transportorder.movetourpoint()	sourcetransportordertix, sourcetourpointid, destinationtourpointid, relationtype, mode
callAddTourpoint	pdis_transportorder.addtourpoint()	p_transportorderid, p_tourpointtype, p_producttype, p_deliverydatefrom/to, p_deliverytimefrom/to, p_tix, p_personnumber, p_name1, p_tournumber, p_referencetourpointid, p_country, p_reference, p_postalcode, p_city, p_streetand housenumber, p_housenumberaddition, p_district, p_tourpointposition → Output: p_new_tourpoint_tix
callEditTourpoint	pdis_transportorder.edittourpoint()	p_tourpointid, p_tix, p_personnumber, p_name1, p_tournumber, p_referencetourpointid, p_country, p_reference, p_postalcode, p_city, p_streetand housenumber, p_housenumberaddition, p_district, p_tourpointposition
callDeleteTourpoint	pdis_transportorder.deletetourpoint()	transportorderid, tourpointid, mode
callSetLoadingReference	pdis_tourpoint.setloadingreference()	tourpointid, loadingreference
callSetTargetLoadingStartTime	pdis_tourpoint.settargetloadingstarttime()	tourpointid, dstart
callSetTargetLoadingEndTime	pdis_tourpoint.settargetloadingendtime()	tourpointid, dend
callSetLoadingInterval	pdis_tourpoint.setloadinginterval()	tourpointid, dstart, dend

GraphQL Mutation	Datenbank-Prozedur	Parameter
callRemoveLoadingIntervals	pdis_tourpoint.removeloadingintervals()	tourpointid
callSetCustomerTourNumber	pdis_tourpoint.setcustomertournumber()	tourpointId, tourNumber
callStaysLoaded	pdis_leg.staysloaded()	legid, staysloadedflag

7.5 Mutations → Datenbank-Prozeduren (Transportauftrag-Management)

GraphQL Mutation	Datenbank-Prozedur
callCreateTransportOrderFromLeg	pdis_transportorder.createtransportorderfromleg()
callCreateTransportOrderFromLot <i>(obsolete)</i>	pdis_transportorder.createtransportorderfromshipment()
callCreateAndAddLeg	pdis_transportorder.createandaddleg()
callDeleteTransportOrder	pdis_transportorder.delete()
callRemoveLeg	pdis_transportorder.removeleg()
callRemoveShipmentFromTransportOrder	pdis_transportorder.removeshipment()
callAssignLotToTransportOrder <i>(obsolete)</i>	pdis_transportorder.addshipment()
callAssignVehicle	pdis_transportorder.addvehicle()
callAddTrailer	pdis_transportorder.addtrailer()
callSetVehicleAttributes	pdis_transportorder.setvehicleattributes()
callSetPresetTemp	pdis_transportorder.setpresettemp()
callSetParticipant	pdis_transportorder.setparticipant()
callSetXServerDto	pdis_transportorder.setxserverdto()

7.6 MDE-Mutations (Mobile Data Entry)

GraphQL Mutation	Datenbank-Prozedur
callAbschInveProcedure (DispMdeAh)	disp_mde_ah.abschInve()
callStartEntladungProcedure (DispMdeAh)	disp_mde_ah.startentladung()
callEndeEntladungProcedure (DispMdeAh)	disp_mde_ah.endeentladung()
callScanBarcodeProcedure (DispMdeAh)	disp_mde_ah.scanbarcode()
callAbschInveProcedure (DispMdeEb)	disp_mde_eb.abschInve()
callEndeEntladungProcedure (DispMdeEb)	disp_mde_eb.endeentladung()

8. Quellcode-Referenzen

Backend (Disposition-Backend)

Bereich	Pfad
Controllers	Features/PickupPlanning/PickupPlanningController.cs , Features/TransportOrders/TransportOrdersController.cs
Drive Instructions	Features/PickupPlanning/Requests/GetDriveInstructions/
Tourpoint Reorder	Features/PickupPlanning/Requests/ReorderTourpoint/
Tourpoint Edit	Features/TransportOrders/Requests/EditTourpoint/
Leg Reorder	Features/PickupPlanning/Requests/ReorderLeg/
Lot Assignment Reorder	Features/PickupPlanning/Requests/ReorderLotAssignment/
Stays Loaded	Features/PickupPlanning/Requests/MarkLegStaysLoaded/
Loading Reference	Features/TransportOrders/Requests/SetTourpointLoadingReference/
Loading Interval	Features/TransportOrders/Requests/SetTourpointLoadingInterval/
Auth Config	Infrastructure/ServiceSetupExtensions/KeyCloack/
Swagger Config	Infrastructure/ServiceSetupExtensions/Swagger/
Domain Entities	Domain/Entities/LotAssignment/ , Domain/Entities/Leg/ , Domain/Entities/LotAssignmentLegLink/

TMS Bridge (Disposition-Abstraction-Layer)

Bereich	Beschreibung
GraphQL Queries	Queries/ — pro Entität eine Query-Klasse (z.B. TransportOrderQuery.cs)
GraphQL Mutations	Mutations/ — pro Operation eine Mutation-Klasse (z.B. MoveTourpointMutation.cs)
DbContext	BranchDbContext.cs — Entity Framework Core mit DbSet-Mappings auf Views und Tabellen
Stored Procedure Calls	Via IRoutineExecutor mit OperationType.Procedure oder OperationType.Function

Frontend (Disposition-Frontend)

Bereich	Pfad
Services	apps/nagel-cal-disposition/src/app/services/
Models	apps/nagel-cal-disposition/src/models/
Endpoints	apps/nagel-cal-disposition/src/app/configuration/consts/endpoints.ts
Utils	apps/nagel-cal-disposition/src/app/utils/tourPointsUtils.ts
Keycloak	libs/nagel-services/src/lib/keycloakService/keycloak.service.ts
Request Service	libs/nagel-services/src/lib/requestService/request.service.ts

Architektur-Hinweise

- **CQRS-Pattern** — Alle Operationen sind in Query-Handler (Lesen) und Command-Handler (Schreiben) aufgeteilt.
- **GraphQL-Integration** — Schreiboperationen (Tourpunkte verschieben/bearbeiten/anlegen/löschen, Stays Loaded) werden an die TMS Bridge via GraphQL-Mutations delegiert.
- **Lokale DB-Operationen** — Leg-Umsortierung und Lot-Assignment-Umsortierung schreiben direkt nach PostgreSQL via Entity Framework.
- **Validierung** — Dedizierte `ICommandValidator` / `IQueryValidator` pro Operation.
- **Dual-DB-Support** — Das System unterstützt sowohl Oracle- als auch PostgreSQL-Datenbanken (dialekt-spezifisches Handling im `BranchDbContext`).